

1 PART 1: Pandas Foundations & Data Preprocessing

1.1 The Golden Rule of sklearn

KEY CONCEPT

fit on TRAIN, transform on TEST

- `fit(X_train, y_train)` — Learn parameters from training data
- `transform(X_train)` — Apply to training data
- `transform(X_test)` — Apply SAME parameters to test data
- `fit_transform(X_train)` — Shortcut for `fit` + `transform` on TRAIN only

COMMON EXAM TRAP

NEVER call `fit_transform` on test data! This causes **data leakage** — the test data "leaks" into the learned parameters, artificially inflating your scores.

1.2 Pandas: loc vs iloc — The Most Common Trap

Method	Selection by	Slicing Rule
<code>df.loc[r,c]</code>	Labels (index names, column names)	Inclusive (includes end)
<code>df.iloc[r,c]</code>	Integer positions (0, 1, 2, ...)	Exclusive (Python rule)
<code>df['col']</code>	Column name	—

Memorize this example:

```
1 import pandas as pd
2 df = pd.DataFrame({'A': [10,20,30,40], 'B': [50,60,70,80]})
3
4 df.loc[0:2]      # Returns rows 0, 1, 2 -> 3 rows (INCLUSIVE!)
5 df.iloc[0:2]     # Returns rows 0, 1 -> 2 rows (EXCLUSIVE!)
```

1.3 What Returns What?

KEY CONCEPT

- `load_iris()` → **Bunch** object (NOT a DataFrame!)
- `load_iris(as_frame=True)` → `bunch.data` is a DataFrame
- `df['age']` → **Series** (1D)
- `df[['age', 'name']]` → **DataFrame** (2D)
- `df.iloc[:, 0]` → **Series** (single column)
- `df.iloc[:, [0,1]]` → **DataFrame** (multiple columns)

1.4 Data Loading: `load_*` vs `fetch_*` vs `make_*`

Family	Where data lives	Examples
<code>load_*</code>	Bundled with sklearn (small)	<code>load_iris</code> , <code>load_digits</code> , <code>load_wine</code>
<code>fetch_*</code>	Downloaded from internet (large)	<code>fetch_california_housing</code> , <code>fetch_openml</code>
<code>make_*</code>	Generated synthetically	<code>make_classification</code> , <code>make_blobs</code>

💡 **TIP:** All `load_*` and `fetch_*` return a **Bunch** object with `.data`, `.target`, `.feature_names`, and `.target_names`.

1.5 Imputation — Handling Missing Values

```
1 from sklearn.impute import SimpleImputer
2
3 # For numeric columns with outliers -> use median
4 imputer = SimpleImputer(strategy='median')
5 X_train_filled = imputer.fit_transform(X_train)
6 X_test_filled = imputer.transform(X_test) # NO refitting!
```

SimpleImputer Strategies:

- `'mean'` — Fill with column mean (no outliers)
- `'median'` — Fill with column median (robust to outliers)
- `'most_frequent'` — Fill with mode (for categorical data)
- `'constant'` — Fill with a fixed value

1.6 Categorical Encoding — Choose the Right One

KEY CONCEPT

- `OrdinalEncoder` — For ORDERED categories (Low/Medium/High)
- `OneHotEncoder` — For NOMINAL categories (cities, colors) ← Most common
- `LabelEncoder` — For TARGET variable ONLY!

COMMON EXAM TRAP

Don't use `OrdinalEncoder` on unordered categories! The model will incorrectly assume city names have an order, which is nonsensical.

1.7 Feature Scaling — Which Algorithms Need It?

MUST scale

KNN (distance-based)
K-Means (distance-based)
SVM (margin/distance)
PCA (variance-based)
Logistic Regression (regularization)
Neural Networks / MLP
SGDRegressor (gradient descent)

DON'T need scaling

Decision Tree
Random Forest
Naive Bayes
Gradient Boosting (mostly)

Which Scaler to Use?

- `StandardScaler` — Default choice. Mean=0, Std=1. Not outlier-robust.
- `MinMaxScaler` — Output in $[0,1]$. Good for neural nets.
- `RobustScaler` — Uses median & IQR. **BEST for data with outliers.**
- `MaxAbsScaler` — Preserves sparsity. Output in $[-1,1]$.

1.8 Outlier Detection Methods

- **Z-score method:** Keep points where $|z| \leq 3$ (assumes normal distribution)
- **IQR method:** Keep points within $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ (robust, works for skewed data)
- **Percentile capping (Winsorization):** Cap at 1st and 99th percentiles

2 PART 2: Feature Selection, PCA, Pipelines & Regression

2.1 Feature Selection Methods

Type	Method	How it works
Filter	SelectKBest, VarianceThreshold	Uses simple statistics (fast)
Wrapper	RFE (Recursive Feature Elimination)	Trains model multiple times
Embedded	SelectFromModel (Lasso)	Model does selection during training

SelectKBest Scoring Functions:

- `f_classif` — Classification, numeric features (ANOVA F-value)
- `f_regression` — Regression, numeric features
- `chi2` — Classification, **NON-NEGATIVE** features only!
- `mutual_info_classif` — Any features, captures non-linear relationships

COMMON EXAM TRAP

chi2 trap: `chi2` requires features to be NON-NEGATIVE. If you've used `StandardScaler` (produces negative values), `chi2` will throw an error!

2.2 PCA — Principal Component Analysis

KEY CONCEPT

PCA Goal: Reduce dimensions while keeping maximum **variance**.

$$\text{Information preserved} = \sum_{i=1}^k \text{explained_variance_ratio}_i$$

```
1 from sklearn.decomposition import PCA
2
3 pca = PCA(n_components=2) # Keep top 2 components
4 X_reduced = pca.fit_transform(X_scaled) # MUST scale first!
5
6 pca.explained_variance_ratio_ # e.g., [0.72, 0.23] means 95%
   variance kept
7 pca.components_ # The principal directions
```

💡 **TIP: Always scale before PCA!** PCA is variance-based — a feature measured in millions will dominate one measured in 0-1.

2.3 Pipelines — Preventing Data Leakage

KEY CONCEPT

A **Pipeline** chains preprocessing steps + model into one object. It **automatically** prevents the `fit_transform` on test data mistake!

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.impute import SimpleImputer
4 from sklearn.linear_model import LogisticRegression
5
6 pipe = Pipeline([
7     ('imputer', SimpleImputer(strategy='median')),
8     ('scaler', StandardScaler()),
9     ('model', LogisticRegression())
10 ])
11
12 pipe.fit(X_train, y_train)      # Fits all steps in sequence
13 pipe.predict(X_test)           # Only transforms test data    SAFE!
```

2.4 ColumnTransformer — Different Preprocessing for Different Columns

```
1 from sklearn.compose import ColumnTransformer
2 from sklearn.preprocessing import StandardScaler, OneHotEncoder
3
4 numeric_cols = ['age', 'income']
5 categorical_cols = ['city', 'gender']
6
7 preprocessor = ColumnTransformer([
8     ('num', StandardScaler(), numeric_cols),
9     ('cat', OneHotEncoder(), categorical_cols)
10 ], remainder='passthrough') # Keep unlisted columns as-is
```

2.5 Regression Metrics

$$\text{MAE} = \frac{1}{n} \sum |y_{\text{actual}} - y_{\text{pred}}|$$

$$\text{MSE} = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{pred}})^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = 1 - \frac{\sum (y_{\text{actual}} - y_{\text{pred}})^2}{\sum (y_{\text{actual}} - \bar{y})^2}$$

💡 **TIP: R^2 range:** $-\infty$ to 1. $R^2 = 1$ = perfect predictions. $R^2 = 0$ = predicting the mean. R^2 can be **NEGATIVE** (model worse than predicting the mean!).

3 PART 3: Classification — Models & Metrics

3.1 Confusion Matrix — sklearn Convention

MUST MEMORIZE — Rows = ACTUAL, Columns = PREDICTED

$$\text{cm} = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

Where: $\text{cm}[0,0]=TN$, $\text{cm}[0,1]=FP$, $\text{cm}[1,0]=FN$, $\text{cm}[1,1]=TP$

3.2 Key Metrics — Know These Formulas

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Interpretation:

- **Precision:** "Of those I said were positive, how many were right?" — Quality of positives
- **Recall:** "Of all actual positives, how many did I catch?" — Coverage of positives
- **F1:** Harmonic mean — punishes extremes

3.3 ROC-AUC — What You Must Know

KEY CONCEPT

- **ROC Curve:** Plots TPR (Recall) vs FPR at all thresholds
- **AUC = 1.0** → Perfect classifier
- **AUC = 0.5** → Random guessing (diagonal line)
- **AUC interpretation:** Probability that model ranks a random positive sample higher than a random negative sample
- **AUC is threshold-independent** — uses all thresholds, not just 0.5

3.4 Logistic Regression — The Big Trap

COMMON EXAM TRAP

C is the INVERSE of regularization strength!

Smaller C = Stronger regularization

Larger C = Weaker regularization

This is OPPOSITE to α in Ridge/Lasso!

```
1 from sklearn.linear_model import LogisticRegression
2
3 # C=0.01 -> STRONG regularization (simple model)
4 # C=100 -> WEAK regularization (complex model)
5 model = LogisticRegression(C=1.0, penalty='l2')
6 model.predict_proba(X_test) # Get probabilities!
```

3.5 Multi-class Strategies

Strategy	Number of classifiers	Used by
OvR (One-vs-Rest)	n	Logistic Regression (default for some solvers)
OvO (One-vs-One)	$n(n-1)/2$	SVM always uses OvO internally
Multinomial (Softmax)	1	Logistic Regression with <code>multi_class='multinomial'</code>

3.6 KNN — Lazy Learner

KEY CONCEPT

- **Training:** Does NOTHING (just stores data) — FAST
- **Prediction:** Computes distances to all points — SLOW
- **MUST scale features!** Distance-based algorithm
- **Curse of dimensionality:** KNN struggles beyond 10-20 features

3.7 Naive Bayes — No Scaling Needed!

- **GaussianNB** — Continuous features (assumes normal distribution)

- **MultinomialNB** — Count data (text classification, word counts)
- **BernoulliNB** — Binary features (0/1, presence/absence)

💡 **TIP:** Naive Bayes assumes features are independent given the class — ”naive” but works surprisingly well for text classification (spam detection).

3.8 Ensembles — The Ultimate Comparison

Method	How it works	Reduces
Bagging (Random Forest)	Parallel models on bootstrapped data	Variance (overfitting)
Boosting (AdaBoost/GB)	Sequential models fixing prior errors	Bias (underfitting)
Voting	Combine different models (hard/soft)	Both (by averaging)

3.9 Random Forest — The Go-To Default

```

1 from sklearn.ensemble import RandomForestClassifier
2
3 model = RandomForestClassifier(
4     n_estimators=100,          # Number of trees
5     max_features='sqrt',      # Classification default
6     min_samples_leaf=1,
7     random_state=42
8 )
9 model.feature_importances_    # Free feature importance!

```

4 PART 4: Clustering, Tuning & Cross-Validation

4.1 K-Means Clustering

```
1 from sklearn.cluster import KMeans
2
3 km = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state
4           =42)
5 km.fit(X_scaled) # MUST scale features!
6 km.cluster_centers_ # Shape (k, n_features)
7 km.labels_ # Which cluster each point belongs to
8 km.inertia_ # WCSS lower = tighter clusters
```

💡 **TIP:** Use **elbow method** to choose k : plot inertia vs k , look for the "elbow" where inertia stops dropping sharply.

4.2 Cross-Validation — The Right Way

```
1 from sklearn.model_selection import cross_val_score, StratifiedKFold
2
3 # For CLASSIFICATION use StratifiedKFold
4 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
5 scores = cross_val_score(model, X_train, y_train, cv=skf, scoring='
6 accuracy')
7 print(f"Mean CV accuracy: {scores.mean():.3f} (+/- {scores.std():.3f})
8 ")
```

💡 **TIP:** Passing $cv=5$ to `cross_val_score` with a **classifier** automatically uses **StratifiedKFold**.

4.3 GridSearchCV — The Combination Math

```
1 from sklearn.model_selection import GridSearchCV
2
3 param_grid = {
4     'C': [0.1, 1, 10, 100],
5     'kernel': ['linear', 'rbf']
6 }
7 # Total combinations = 4 C's x 2 kernels = 8 combos
8 # Total fits with cv=5 = 8 x 5 = 40 fits
9
10 grid = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy')
11 grid.fit(X_train, y_train)
12 print(grid.best_params_) # Winning parameter combination
13 print(grid.best_score_) # Mean CV score of winner
```

KEY CONCEPT

Combinations = product of all param list lengths
Total fits = Combinations x cv folds

5 PART 5: Recommender Systems & Time Series

5.1 Recommender Systems — Three Approaches

Approach	Uses	Cold Start?
Content-Based	Item metadata (genre, cast)	Solves new item cold start
Collaborative Filtering	User-item interactions only	Suffers from cold start
Hybrid	Both	Best of both

💡 **TIP:** A newly launched movie has no ratings but has metadata → **Content-Based Filtering** is the answer.

5.2 Cosine Similarity — The Formula You Must Compute

$$\text{cosine_similarity}(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \cdot \sqrt{\sum_{i=1}^n b_i^2}}$$

Example: $a = [3, 4, 0]$, $b = [4, 0, 3]$

- Dot product: $3 \times 4 + 4 \times 0 + 0 \times 3 = 12$
- $\|a\| = \sqrt{9 + 16 + 0} = 5$, $\|b\| = \sqrt{16 + 0 + 9} = 5$
- Cosine similarity = $12 / (5 \times 5) = 0.48$

5.3 Time Series — Stationarity & ADF Test

KEY CONCEPT

Stationarity: Statistical properties (mean, variance) don't change over time.

ADF Test Interpretation:

- **H (null hypothesis):** Series is NON-stationary (has unit root)
- **p-value < 0.05:** Reject H → Series IS stationary
- **p-value > 0.05:** Fail to reject H → Series is non-stationary

```
1 from statsmodels.tsa.stattools import adfuller
2
3 result = adfuller(series)
4 p_value = result[1]
5
6 if p_value < 0.05:
7     print("Stationary")
```

```
8 else:
9     print("Non-stationary      needs differencing")
```

5.4 Making Data Stationary — Differencing

```
1 # First-order differencing      removes linear trend
2 series_diff = series.diff().dropna()
3
4 # Second-order differencing      for curved trends
5 series_diff2 = series.diff().diff().dropna()
6
7 # Seasonal differencing (e.g., monthly data with yearly seasonality)
8 series_seasonal_diff = series.diff(periods=12).dropna()
```

6 FINAL CHEAT SHEET — Top 10 Exam Traps

COMMON EXAM TRAP

1. **fit_transform on test data** — Never! Causes data leakage.
2. **df.loc vs df.iloc slicing** — loc INCLUSIVE, iloc EXCLUSIVE.
3. **LogisticRegression C parameter** — Smaller C = STRONGER regularization.
4. **KNN bottleneck** — PREDICTION is slow, training is fast.
5. **Scaling needs** — Trees & Naive Bayes DON'T need scaling.
6. **HAC bottom-up** — Doesn't need k upfront.
7. **warm_start=True** — For "continue training from previous weights".
8. **random.state** — Required for reproducibility.
9. **Soft voting** — Averages probabilities, not "max sum".
10. **Encoders** — OneHotEncoder (nominal), OrdinalEncoder (ordered), LabelEncoder (target).

7 Default Scoring Metrics

- **Classifiers:** `.score()` returns **accuracy**
- **Regressors:** `.score()` returns **R^2**
- **KMeans:** returns **negative inertia**

8 Algorithm Quick Selection Guide

You want...	Reach for...
Simple, interpretable regression	<code>LinearRegression</code>
Regression with feature selection	<code>Lasso</code>
Regression with correlated features	<code>Ridge</code>
Probabilistic binary classifier	<code>LogisticRegression</code>
Quick non-parametric classifier	<code>KNeighborsClassifier</code>
Text classification (counts)	<code>MultinomialNB</code>
Best general-purpose without tuning	<code>RandomForestClassifier</code>
Best with tuning & complex data	<code>GradientBoostingClassifier</code>
Highly interpretable rules	<code>DecisionTreeClassifier</code>
Cluster when you know k	<code>KMeans</code>
Cluster without knowing k	<code>AgglomerativeClustering</code>
Reduce dimensions for visualization	<code>PCA(n_components=2)</code>
Preprocess heterogeneous columns	<code>ColumnTransformer</code>

9 Your 3-Day Action Plan

- **Day 1:** Master Part 1 & Part 2 — The Golden Rule, loc/iloc, scaling table, Pipeline
- **Day 2 (Critical):** Master Part 3 — Confusion matrix, Precision/Recall/F1, ROC-AUC, Bagging vs Boosting
- **Day 3:** Review Part 4 (clustering, CV, tuning math) and Part 5 (recommenders, time series)
- **Day before exam:** Practice PYQs — identify which concept each question is testing

Good luck on your MLP exam!

This document contains everything from the 116-page guide — study this, and you're prepared.